Machine Learning

First Assignment

Roufos Dimitrios, 5090

Charalampos Sapranidis, 4969 (Economics)

Packages:

- tabulate
- torch
- torchvision
- sklearn
- matplotlib
- PILLOW (PIL)
- pandas

Comparison of pipelines and models

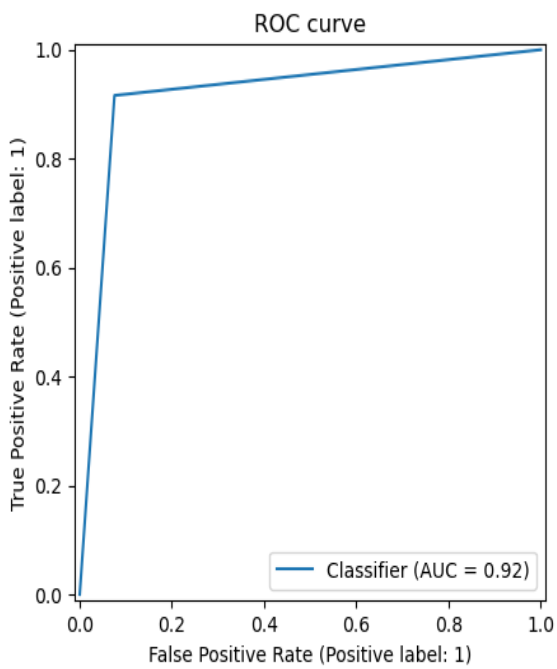
We experimented on two distinct pipelines for the task of recognizing the expression of a face and categorizing it into Pleasant (category 1) and Unpleasant (category 0).

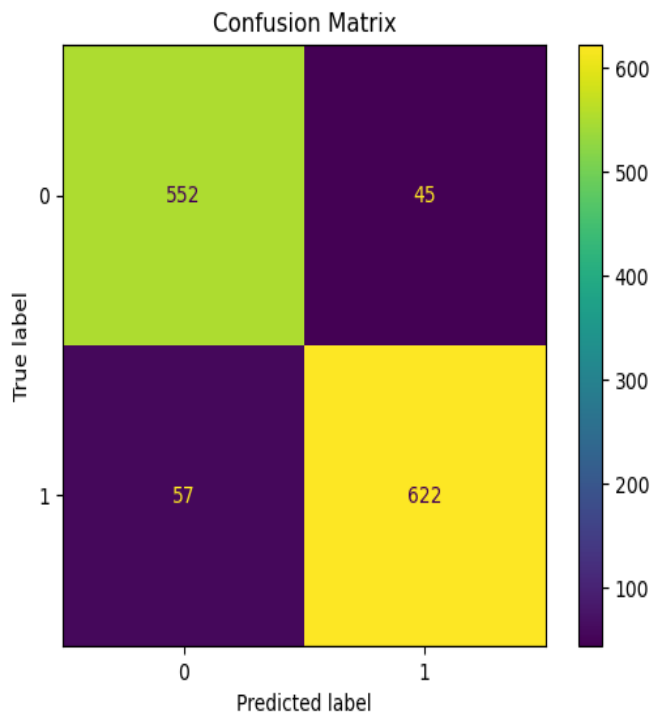
In the first pipeline, we defined a Convolution Neural Network and trained it directly in the image datasets. In the second pipeline, first, we passed the data through the pretrained ResNet CNN, after we had removed the classification layer and we extracted the feature vectors that were used to train traditional Machine Learning models.

For training, we split the data into 2 sets, test and validation. In the first pipeline, training data consisted of 80% of total data, while validation data consisted of 20%. In the second pipeline, training data comprised 90% of total data, while validation data comprised 10%.

1. <u>CNN trained directly on the data</u>

	Metrics	Scores
0	f1	0.9242
1	accuracy	0.9201
2	precision	0.9325
3	recall	0.9161



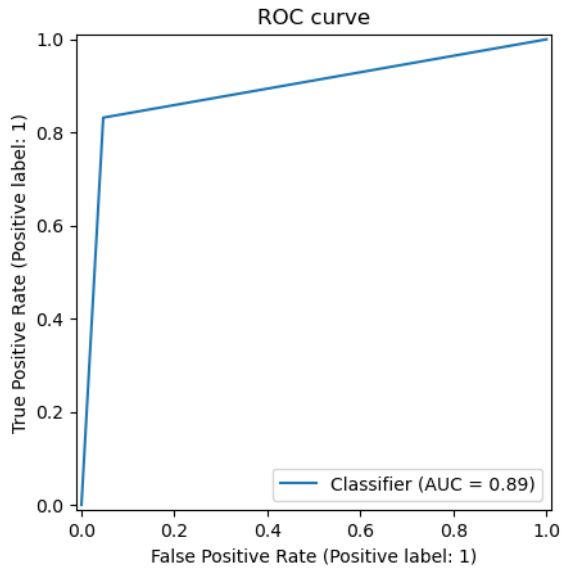


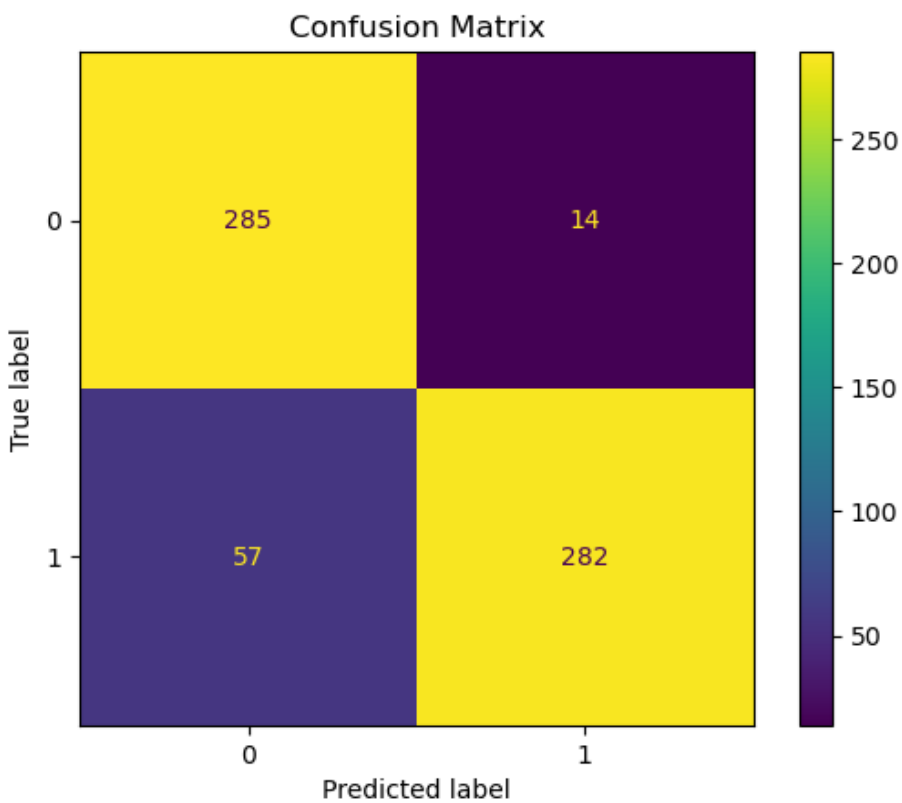
2. Models trained on feature vectors, extracted from ResNet

- K-NN

```
Optimal K-NN Hyperparameters:
                n = 3
    dist. metric = manhattan

+----+-----------+----------+
|    | Metrics   |   Scores |
|----+-----------+----------|
|  0 | f1        |   0.8882 |
|  1 | accuracy  |   0.8887 |
|  2 | precision |   0.9527 |
|  3 | recall    |   0.8319 |
+----+-----------+----------+
```





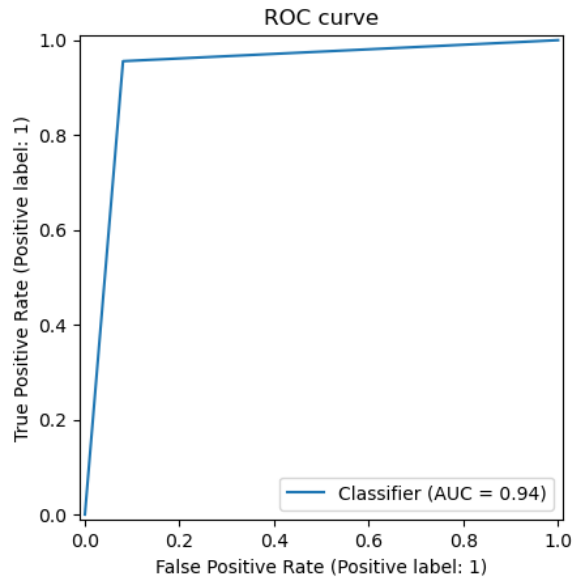
- <u>Multi-Layer Perceptron</u>

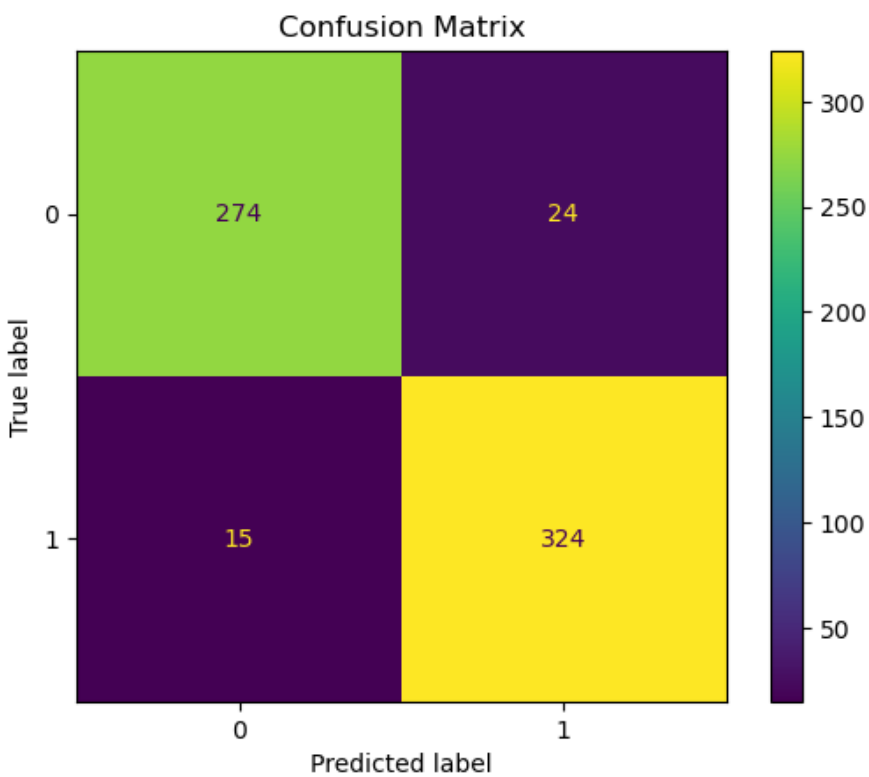
Optimal Parameters:

```
MLPClassifier(hidden_layer_sizes=(512, 512, 512, 512), max_iter=500,
              solver='sgd')
```

Activation function: ReLu

	Metrics	Scores
0	f1	0.9432
1	accuracy	0.9388
2	precision	0.931
3	recall	0.9558



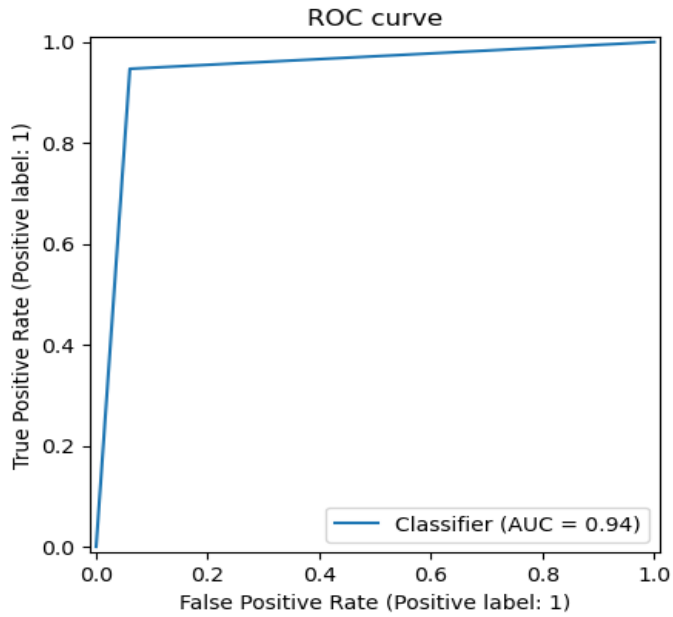


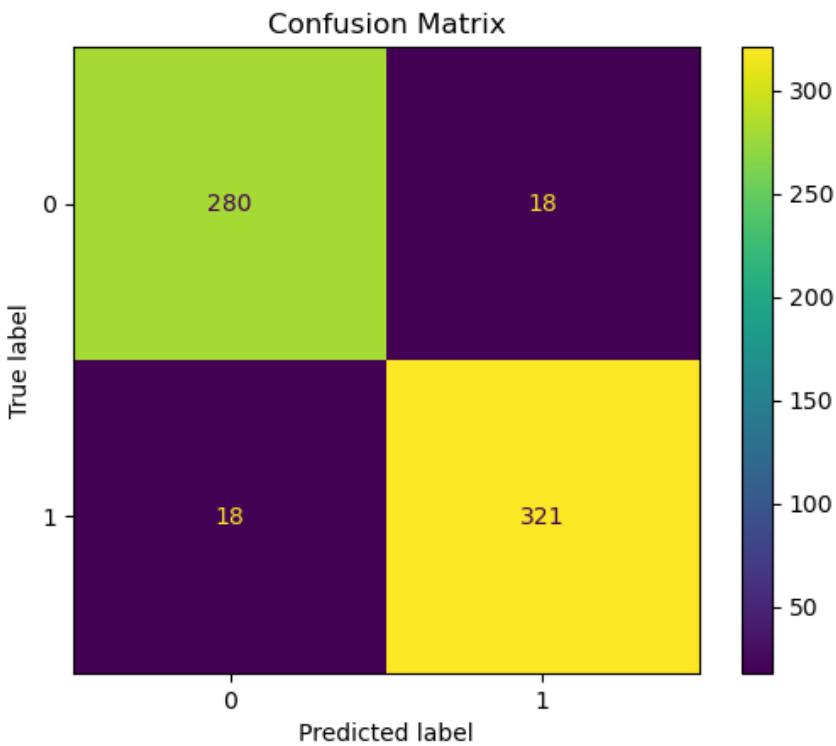
- <u>Support Vector Machine</u>

```
----C value: 0.1
   f1 score: 0.9184
----C value: 1
   f1 score: 0.9419
----C value: 10
   f1 score: 0.9469
----C value: 100
   f1 score: 0.9461

Optimal C value: 10
```

	Metrics	Scores
0	f1	0.9469
1	accuracy	0.9435
2	precision	0.9469
3	recall	0.9469





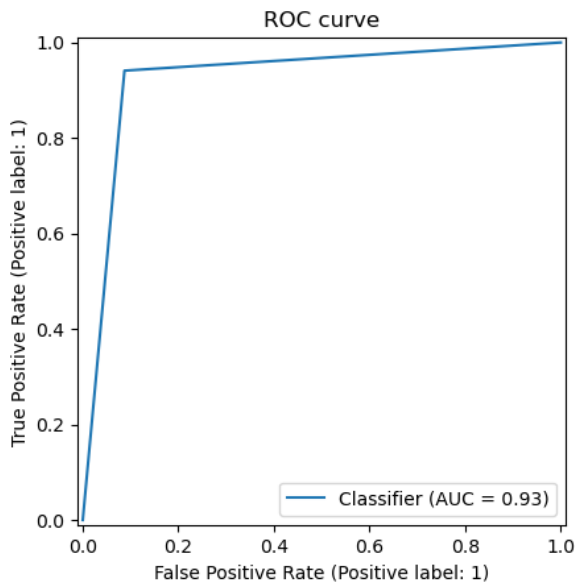
- <u>Logistic Regression</u>

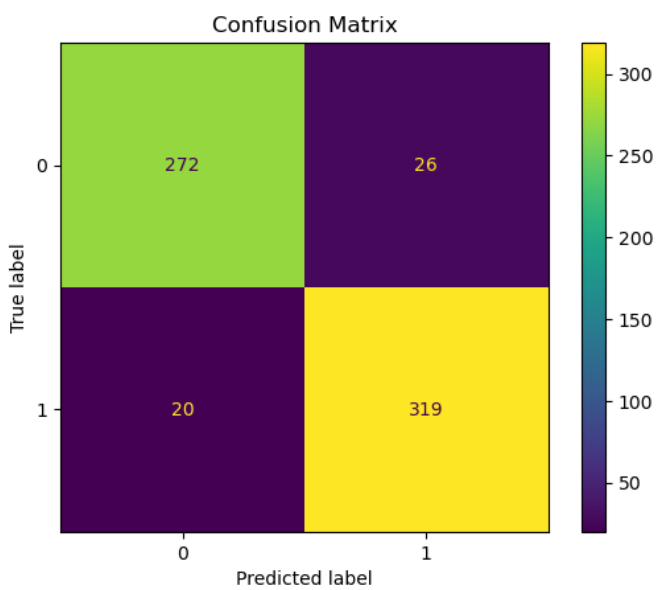
```
----C value: 0.1
   f1 score: 0.9407
----C value: 1
   f1 score: 0.9453

----C value: 10
   f1 score: 0.9370
----C value: 100
   f1 score: 0.9327

Optimal C value: 1
```

	Metrics	Scores
0	f1	0.9327
1	accuracy	0.9278
2	precision	0.9246
3	recall	0.941



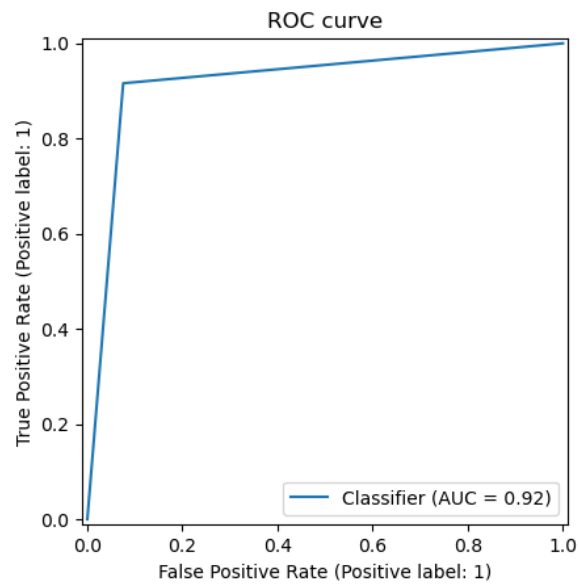


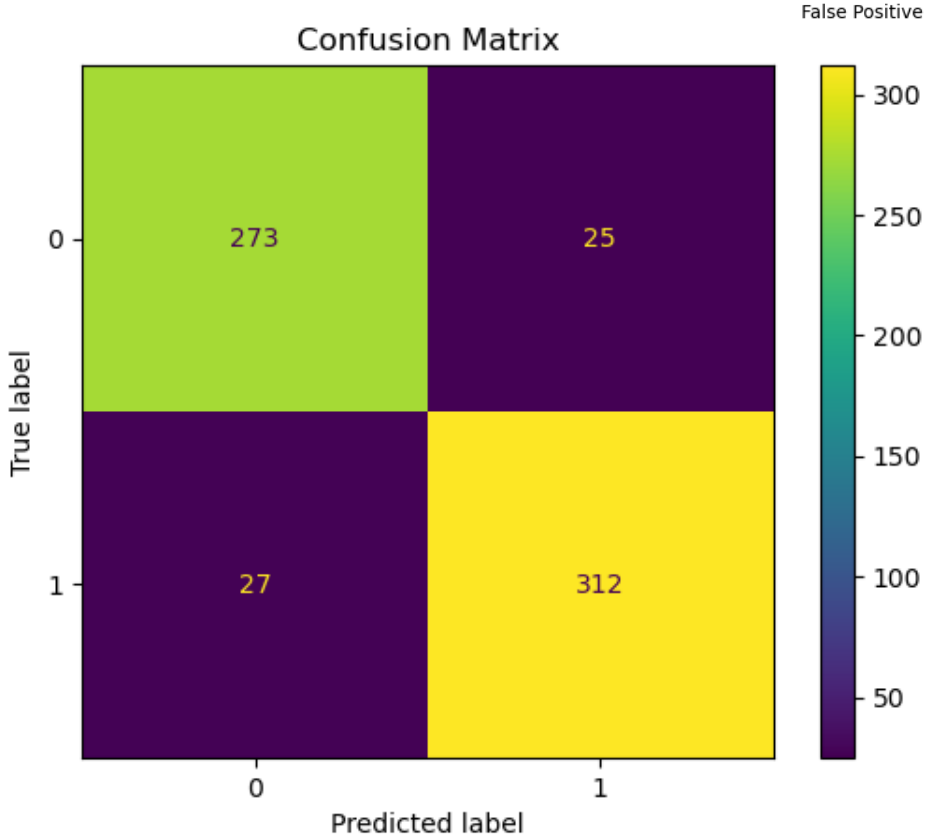
- <u>Ensemble Learning</u>

<u>Random Forest</u>

```
Optimal Random Forest Hyperparameters:
       n = 500
   depth = 50
```

	Metrics	Scores
0	f1	0.9231
1	accuracy	0.9184
2	precision	0.9258
3	recall	0.9204

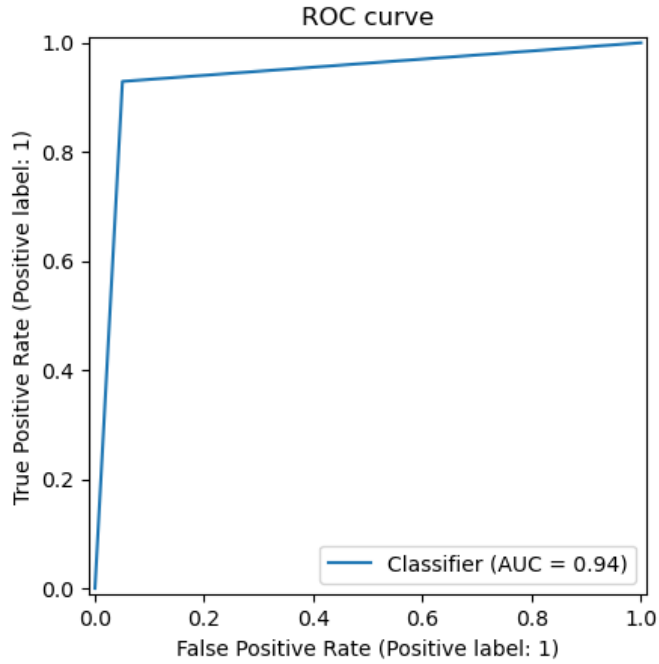


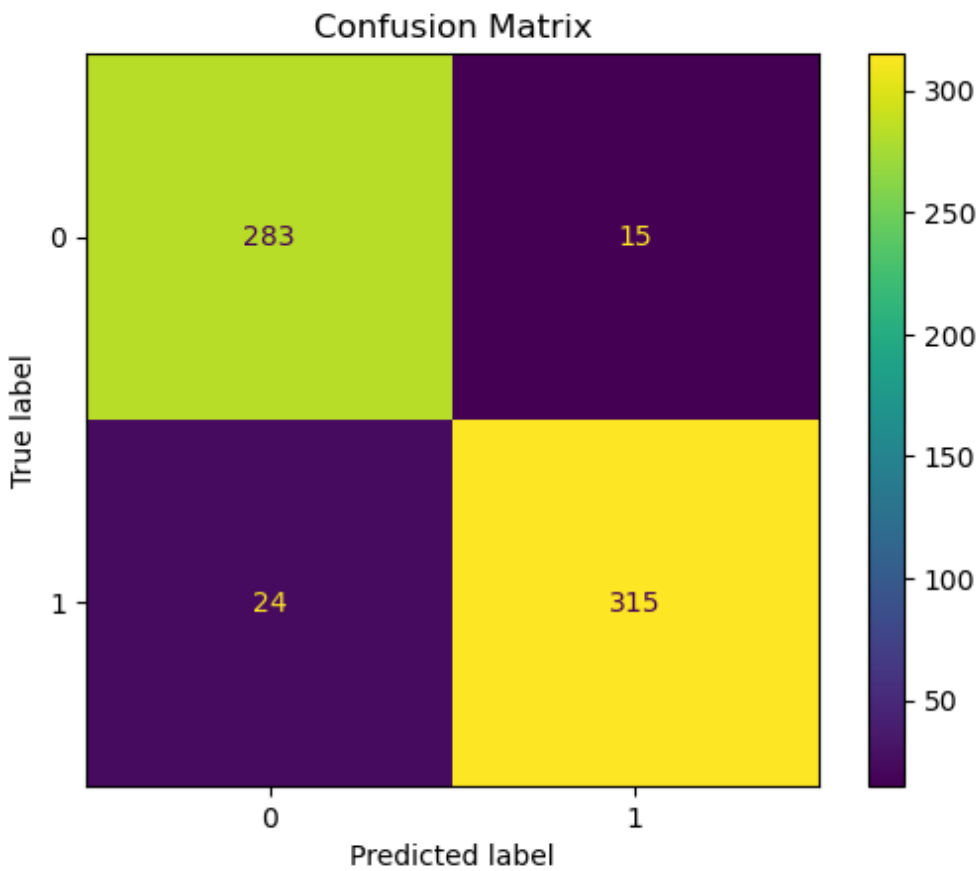


Bagging

```
Optimal Bagging Hyperparameters:
       n = 10

+----+-----------+----------+
|    | Metrics   |   Scores |
|----+-----------+----------|
|  0 | f1        |   0.9417 |
|  1 | accuracy  |   0.9388 |
|  2 | precision |   0.9545 |
|  3 | recall    |   0.9292 |
+----+-----------+----------+
```

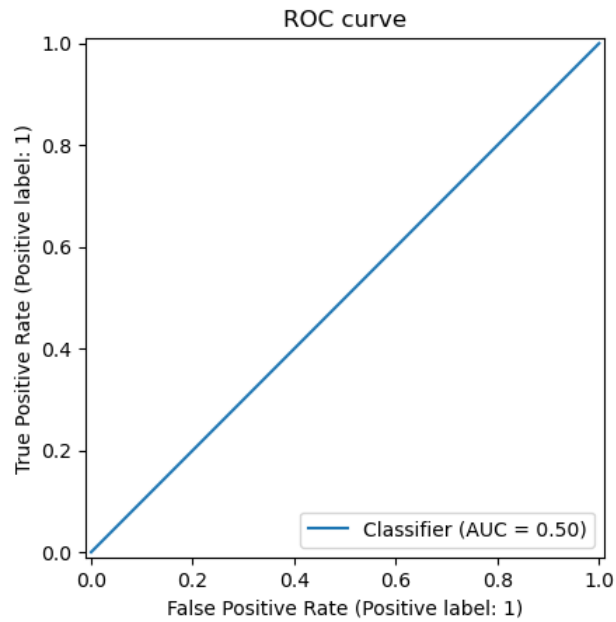


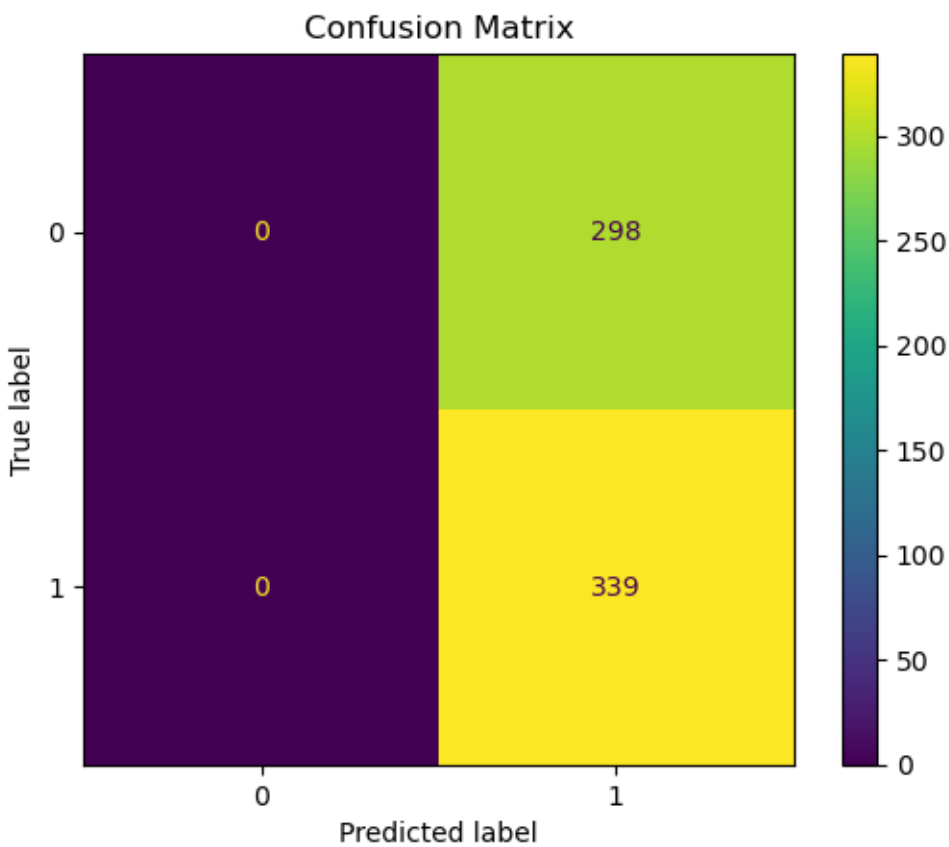


AdaBoost

```
Optimal AdaBoost Hyperparameter:
       n = 100

+----+-----------+----------+
|    | Metrics   |   Scores |
|----+-----------+----------|
|  0 | f1        |   0.6947 |
|  1 | accuracy  |   0.5322 |
|  2 | precision |   0.5322 |
|  3 | recall    |   1      |
+----+-----------+----------+
```





for some reason AdaBoost doesn't work right.

Conclusions and observations

Both pipelines performed quite well, but some models trained on the feature space outperformed the CNN trained on plain images. The K-NN classifier had the poorest of the acceptable results. The MLP had the highest recall and the Bagging classifier had the highest precision. The Support Vector Machine showed the best overall performance, with every metric reaching a 94% score. Logistic Regression performed slightly better than the CNN, but Random Forest performed slightly worse that the CNN. AdaBoost showed poor performance, classifying every image as Pleasant (1) and for that reason the recall is a perfect 1, making it's performance unacceptable.
The overall results, excluding K-NN and AdaBoost are almost similar, so the criterion for choice would be the time needed to train a model, which wasn't taken into consideration.